

Code Explanation

Batch Processing Pipeline Explanation

The given code is a batch processing pipeline implemented using Apache Spark and Delta Lake. It reads customer and order data from CSV files, cleans and processes the data, and then creates or updates two tables: `ordersummary` and `customeraggregatespend`.

Overview of the Code

The code is structured into several functions, each responsible for a specific task in the data processing pipeline. The main function orchestrates the execution of these tasks. The pipeline involves reading source data, cleaning the data, creating or updating the `ordersummary` table with SCD Type 2 implementation, and creating the `customeraggregatespend` table with aggregated data.

Step 1: Creating a Spark Session

The first step is to create a Spark session, which is the entry point to any Spark functionality. This is achieved through the `create\_spark\_session` function.

```
def create_spark_session():
    return SparkSession.builder
        .appName("Customer Order Processing")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Reading Source Data

The next step involves reading the customer and order data from CSV files using the `read\_source\_data` function.

```
def read_source_data(spark):
    customer_df = spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata")

    order_df = spark.read.format("csv")
        .option("header", "true")
        .option("inferSchema", "true")
        .load("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata")

    return customer_df, order_df
```

Step 3: Cleaning the Data

The `clean\_data` function is used to clean the customer and order data by removing duplicates and null values.

```

def clean_data(customer_df, order_df):
    clean_customer_df = customer_df.dropDuplicates(["CustId"])
        .filter(F.col("CustId").isNotNull() &
                F.col("Name").isNotNull() &
                F.col("EmailId").isNotNull() &
                F.col("Region").isNotNull())

    clean_order_df = order_df.dropDuplicates(["OrderId"])
        .filter(F.col("OrderId").isNotNull() &
                F.col("ItemName").isNotNull() &
                F.col("PricePerUnit").isNotNull() &
                F.col("Qty").isNotNull() &
                F.col("Date").isNotNull() &
                F.col("CustId").isNotNull())
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))

    return clean_customer_df, clean_order_df

```

Step 4: Creating or Updating the ordersummary Table

The `create\_ordersummary` function is responsible for creating or updating the `ordersummary` table with SCD Type 2 implementation.

```

def create_ordersummary(spark, customer_df, order_df):
    # Join customer and order data
    joined_df = order_df.join(customer_df, "CustId", "inner")

    # Select columns as per the required schema
    current_data = joined_df.select(
        "CustId", "Name", "EmailId", "Region",
        "OrderId", "ItemName", "PricePerUnit", "Qty", "Date"
    )

    # Check if table exists
    table_exists = spark._jsparkSession.catalog().tableExists("gen_ai_poc_databricksco.sdlc_wizard.ordersummary")

    if table_exists:
        # Implement SCD Type 2 logic
        target_table = DeltaTable.forName(spark, "gen_ai_poc_databricksco.sdlc_wizard.ordersummary")

        # Add hash to detect changes
        current_data = current_data.withColumn(
            "row_hash",
            F.sha2(
                F.concat_ws("||",
                    F.col("Name"),
                    F.col("EmailId"),
                    F.col("Region"),
                    F.col("ItemName"),
                    F.col("PricePerUnit"),
                    F.col("Qty"),
                    F.col("Date")),
                256
            )
        )
    )

```

```

        # Update the target table
        target_table.alias("target").merge(
            updates.alias("updates"),
            "target.CustId = updates.CustId AND target.OrderId = updates.OrderId AND target.IsActive = true"
        ).whenMatched().updateExpr({
            "IsActive": "false",
            "EndDate": "current_timestamp()"
        }).whenNotMatched().insertAll().execute()

    else:
        # Create new table if it doesn't exist
        result_df = current_data
            .withColumn("StartDate", F.current_timestamp())
            .withColumn("EndDate", F.lit(None).cast("timestamp"))
            .withColumn("IsActive", F.lit(True))

        result_df.write.format("delta")
            .mode("overwrite")
            .option("overwriteSchema", "true")
            .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.order_summary")

```

Step 5: Creating the customeraggregatespend Table

The `create\_customeraggregatespend` function creates the `customeraggregatespend` table with aggregated data.

```

def create_customeraggregatespend(spark, order_df):
    # Create table if not exists
    spark.sql("""
        CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.
customeraggregatespend (
            Name STRING,
            TotalAmount DOUBLE,
            Date STRING
        )
        USING DELTA
    """)

    # Read ordersummary table
    ordersummary_df = spark.table("gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary")

    # Join with order data to get TotalAmount
    joined_df = ordersummary_df.join(
        order_df,
        ["OrderId"],
        "inner"
    )

    # Aggregate TotalAmount by Name and Date
    aggregated_df = joined_df.groupBy("Name", "Date")
        .agg(F.sum("TotalAmount").alias("TotalAmount"))

    # Write to customeraggregatespend table

```

```
    aggregated_df.write.format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.customera
aggregatespend")
```

Step 6: Main Function

The `main` function orchestrates the execution of the data processing pipeline.

```
def main():
    spark = create_spark_session()

    # Read source data
    customer_df, order_df = read_source_data(spark)

    # Clean data
    clean_customer_df, clean_order_df = clean_data(customer_df, orde
r_df)

    # Create or update ordersummary table
    create_ordersummary(spark, clean_customer_df, clean_order_df)

    # Create customeraggregatespend table
    create_customeraggregatespend(spark, clean_order_df)

    print("Data processing completed successfully")

if __name__ == "__main__":
    main()
```